

Unit IV: Software Testing Fundamentals:

What is software testing, Why it is important? Verification, Validation, Qualification, STLC, Testing Basics: Levels: Unit, Integration, System, Acceptance, Types: Manual, Automation, functional, non-functional, Techniques: Black-box, White-box, Grey-box, Test Case Design: Techniques: Boundary Value Analysis, Equivalence Partitioning, Writing effective test plans and test cases

Unit IV: Software testing fundamentals Sort Summary =>

Software testing is the process of checking to find error and verify it works according to requirements.

Verification is the process of checking whether software is being developed correctly according to requirements.
"Are we building the software correctly?"

Validation is the process of checking whether the final software meets to user requirements.
"Are we building the correct software?"

Qualification is the process of confirming that software is ready and suitable for use.

STLC (Software testing life cycle): systematically and efficiently follow step-by-step testing process for software testing is called STLC

1. Requirement Analysis - Tester studies requirements carefully.
2. test planning - tester create a testing plan. what, how test
3. test case development - tester write test cases.
4. test environment setup - preparing environment for testing.
5. test execution - now actual testing starts.
6. test closure - tester make final report, bug summary, result

Levels of Testing

1. Unit Testing (First check small module) - is testing individual small components/modules of software.
2. Integration Testing (then connected modules) - checks whether combined modules work properly together.
3. System Testing (then check full software) - tests the complete integrated software system.
4. Acceptance Testing (then customer checks) - checks whether software is ready for deliver and satisfies customer requirements.

Types of Testing

1. Manual Testing - testing software manually without using automation tools.
2. Automation Testing - use tools/scripts to test software automatically. (Selenium, TestNG, JUnit)
3. Functional Testing - checks whether software functions work according to requirements. "Is software doing correct work?"

4. Non-Functional Testing - checks Performance, Security, Usability, Reliability (all quality factors.) "How well software work?"

Testing Techniques

1. Black-box Testing (outer check) - tests software functionality without knowing internal code.
2. White-box Testing (inner check) - tests internal code structure and programming logic.
3. Grey-box Testing (combine both) - testing with partial knowledge of internal code.

Test Case Design Techniques - When tester test software, they create test cases. But testing every possible input is very difficult. So tester use smart techniques.

1. Boundary Value Analysis (BVA) - testing technique where testing is done on minimum and maximum limit values.
2. Equivalence Partitioning (EP) - divides input data into valid and invalid groups for testing

Writing Effective Test Plans and Test Cases

1. Test Plans - Document describing testing strategy, objectives, schedule and resources.
2. Test Cases - set of steps used to check a feature.

Topic 1: What is Software Testing?

Software Testing means:

👉 Checking software/application to find errors, bugs, or problems before giving it to users.

Simple meaning:

“We test software to check:

- Is it working correctly?
- Is it giving correct output?
- Is it safe and easy to use?”

Real Life Example

Suppose you make a calculator app.

You test:

- $2 + 2 = 4$ or not?
- App crashes or not?
- Buttons work or not?

This checking process is called **Software Testing**.

Why Software Testing is Important?

Without testing:

- Software may crash
- Wrong output may come
- User data may be lost
- Security problems can happen

So testing helps to:

- Find bugs early
 - Improve quality
 - Make software stable
 - Make users happy
-

Small Real Example

Imagine:

You book movie tickets online.

If testing is not done properly:

- Money deducted twice
- Seat not booked
- App crashes after payment

That is why testing is very important.

Main Goal of Testing

Testing checks:

- ✓ Software works correctly
 - ✓ Software is fast
 - ✓ Software is secure
 - ✓ Software has fewer bugs
-

Important Terms

Term	Simple Meaning
Bug	Mistake/problem in software
Error	Wrong coding by programmer
Failure	Software not working properly
Tester	Person who tests software

Short Definition (Exam Style)

Software Testing is the process of checking software to find errors and verify that it works according to requirements.

Very Short Summary

- Testing = checking software
 - Purpose = find bugs/problems
 - Helps improve quality and reliability
-

Write **"next"** when you want the next topic:

👉 **Verification, Validation, Qualification**

next

Topic 2: Verification, Validation, Qualification

These three words are very important in Software Testing.

Do not worry — they look difficult, but are actually simple.

1. Verification

Simple Meaning

Verification checks:

👉 “Are we building the software correctly?”

It checks:

- Design
- Documents
- Code logic
- Requirements

Mostly done **before running software**.

Easy Real Example

Teacher checks your exam paper format before you submit.

Like:

- Name written?
- Roll number written?
- Correct format?

This is similar to verification.

Verification Focus

It checks:

- ✓ Process
 - ✓ Documents
 - ✓ Design
 - ✓ Coding mistakes early
-

Important Point

- ✗ Software may not run here
 - ✓ Mostly checking on paper/code/design
-

Short Definition

Verification is the process of checking whether software is being developed correctly according to requirements.

2. Validation

Simple Meaning

Validation checks:

👉 “Are we building the correct software?”

Here we actually run software and test it.

Easy Real Example

You made a food delivery app.

Validation checks:

- Can user order food?
- Payment works?
- Correct restaurant opens?

If user needs are satisfied → validation successful.

Validation Focus

It checks:

- ✅ Final product
 - ✅ User needs
 - ✅ Actual working software
-

Important Point

✓ Software is executed/run here

Short Definition

Validation is the process of checking whether the final software meets user requirements.

Difference Between Verification and Validation

Verification	Validation
Checks process	Checks final product
Done before execution	Done after execution
“Building software correctly?”	“Building correct software?”
Static checking	Dynamic checking

3. Qualification

Simple Meaning

Qualification means:

👉 Checking whether software is ready for real use.

It confirms:

- Software quality is acceptable
 - Requirements are satisfied
 - Software can be released
-

Real Example

Before launching banking software:

- Security tested
- Speed tested
- Functions tested

If everything is okay → software is qualified for release.

Short Definition

Qualification is the process of confirming that software is ready and suitable for use.

Very Easy Memory Trick

Word	Easy Question
Verification	Are we building it correctly?
Validation	Are we building the right product?
Qualification	Is software ready to use?

Small Exam Answer (2-3 lines)

Verification

Checking software development process without executing program.

Validation

Checking actual software by running it to meet user needs.

Qualification

Confirming software is ready for real-world use.

Write "next" for next topic:

👉 **STLC (Software Testing Life Cycle)**

next

Topic 3: STLC (Software Testing Life Cycle)

STLC means:

Software Testing Life Cycle

It is a step-by-step process followed during software testing.

Simple meaning:

- 👉 Testing is not done randomly.
- 👉 Testers follow proper stages/steps.

These stages together are called **STLC**.

Why STLC is Important?

STLC helps to:

- Do testing properly
 - Save time
 - Find bugs early
 - Improve software quality
-

Main Phases of STLC

There are mainly 6 phases:

No	Phase Name
1	Requirement Analysis
2	Test Planning
3	Test Case Development
4	Test Environment Setup
5	Test Execution
6	Test Closure

1. Requirement Analysis

Simple Meaning

Tester studies requirements carefully.

They understand:

- What software should do
 - Which features need testing
-

Example

For login page:

- Username required?
- Password required?
- Forgot password available?

Tester studies all this.

2. Test Planning

Simple Meaning

Here tester creates a testing plan.

Plan includes:

- What to test
 - How to test
 - Who will test
 - Time and cost
-

Example

Manager decides:

- 5 testers needed
 - Testing takes 10 days
-

3. Test Case Development

Simple Meaning

Tester writes test cases.

A test case means:

👉 Steps to test software.

Example Test Case

Step	Expected Result
Enter correct password	Login successful

4. Test Environment Setup

Simple Meaning

Preparing environment for testing.

Like:

- Computer setup
 - Server setup
 - Internet
 - Database
-

Example

Install app on testing computer before testing.

5. Test Execution

Simple Meaning

Now actual testing starts.

Tester:

- Runs test cases

- Finds bugs
 - Reports bugs
-

Example

Tester clicks login button.

If app crashes → bug found.

6. Test Closure

Simple Meaning

Testing work is completed.

Tester prepares:

- Final report
 - Bug summary
 - Testing result
-

Simple Flow of STLC

```
Requirement Analysis
    ↓
Test Planning
    ↓
Test Case Development
    ↓
Environment Setup
    ↓
Test Execution
    ↓
Test Closure
```

Easy Real Life Example

Think about cooking food:

Cooking Step	STLC Step
Decide recipe	Requirement Analysis
Plan ingredients	Test Planning
Write recipe steps	Test Case Development
Prepare kitchen	Environment Setup
Cook food	Test Execution
Finish and serve	Test Closure

Short Definition (Exam Style)

STLC is a step-by-step testing process followed to perform software testing systematically and efficiently.

Very Short Summary

- STLC = Testing process stages
 - Helps organized testing
 - Contains 6 phases
-

Write "**next**" for next topic:

👉 **Testing Basics: Levels of Testing (Unit, Integration, System, Acceptance)**

Next "you forgot i am asking you to minimum use separator lines on answer, remember this on you memory"

Updated memory

Topic 4: Levels of Testing

Levels of testing means:

👉 Software is tested in different stages/levels.

Mainly 4 levels:

1. Unit Testing
2. Integration Testing
3. System Testing
4. Acceptance Testing

Think like building a car:

- First check small parts
- Then connect parts
- Then check full car
- Then customer checks car

Same happens in software testing.

1. Unit Testing

Simple Meaning

Testing a single small part of software.

A small part is called a **Unit**.

Usually done by programmers.

Example

In calculator app:

- Test only addition function
- Test only subtraction function

Not full app.

Goal

Check whether one small module works correctly.

Short Definition

Unit Testing is testing of individual small components/modules of software.

2. Integration Testing

Simple Meaning

After small units work properly, we connect them and test together.

This is Integration Testing.

Example

Login system connected with database.

Check:

- Username goes to database correctly?
- Password checking works?

Goal

Check communication between modules.

Short Definition

Integration Testing checks whether combined modules work properly together.

3. System Testing

Simple Meaning

Testing the complete full software.

Now entire application is tested.

Example

Testing complete online shopping app:

- Login
- Search product
- Add to cart
- Payment
- Logout

Everything together.

Goal

Check complete system behavior.

Short Definition

System Testing tests the complete integrated software system.

4. Acceptance Testing

Simple Meaning

Final testing done to check whether customer/user accepts software or not.

Usually done before release.

Example

Bank checks banking software before using it officially.

If customer says:

✔ “Yes, this works properly.”

Then software is accepted.

Goal

Check software satisfies user/business needs.

Short Definition

Acceptance Testing checks whether software is ready for delivery and satisfies customer requirements.

Easy Flow

```
Unit Testing
  ↓
Integration Testing
  ↓
System Testing
  ↓
Acceptance Testing
```

Easy Real Life Example

Making a bike:

Testing Level	Bike Example
Unit	Check engine only

Testing Level	Bike Example
Integration	Connect engine + wheels
System	Check full bike
Acceptance	Customer test drive

Difference Table

Level	Checks What?
Unit	Small module
Integration	Connected modules
System	Full software
Acceptance	Customer requirements

Very Short Summary

- Unit → single part
- Integration → connected parts
- System → full software
- Acceptance → customer approval

Write "**next**" for next topic:

👉 **Types of Testing (Manual, Automation, Functional, Non-Functional)**

next

Topic 5: Types of Testing

Software testing has different types.

Main types in your syllabus:

1. Manual Testing
2. Automation Testing
3. Functional Testing
4. Non-Functional Testing

1. Manual Testing

Simple Meaning

Testing software by humans manually.

Tester:

- Clicks buttons
- Enters data
- Checks output

No special automation tool used.

Example

Tester opens login page and checks:

- Correct password works or not
- Wrong password shows error or not

All done manually.

Advantages

- Easy for beginners
- Human observation possible
- Good for small projects

Disadvantages

- Slow
- Time consuming
- Repeating same test is boring

Short Definition

Manual Testing is testing software manually without using automation tools.

2. Automation Testing

Simple Meaning

Testing done using tools or scripts automatically.

Computer performs testing.

Example

A testing tool automatically:

- Opens website
- Clicks login
- Checks result

Without human repeating every step.

Popular Tools

- Selenium
- TestNG
- JUnit

(Just remember names.)

Advantages

- Fast
- Saves time
- Good for repeated testing

Disadvantages

- Tool knowledge needed
- Setup cost higher

Short Definition

Automation Testing uses tools/scripts to test software automatically.

Difference Between Manual and Automation Testing

Manual Testing	Automation Testing
Human performs test	Tool performs test
Slow	Fast
No coding needed	Some coding/tools needed
Good for small testing	Good for repeated testing

3. Functional Testing

Simple Meaning

Checks software functions/features.

Question:

👉 “Is software doing correct work?”

Example

ATM machine:

- Withdraw money works?
- Balance check works?
- PIN check works?

Focus

Checks:

- Input
- Output
- Features
- Functions

Short Definition

Functional Testing checks whether software functions work according to requirements.

4. Non-Functional Testing

Simple Meaning

Checks software quality/performance.

Question:

👉 “How well does software work?”

Example

Check:

- Website speed
- Security
- Performance

- Load handling

Focus

Checks:

- Speed
- Security
- Reliability
- Performance

Short Definition

Non-Functional Testing checks performance, security, usability, and other quality factors.

Easy Example

Food delivery app:

Testing Type	Example
Functional	Order button works?
Non-Functional	App speed good during heavy users?

Very Easy Memory Trick

Type	Main Question
Functional	What software does?
Non-Functional	How well software works?

Very Short Summary

- Manual → human testing
- Automation → tool testing
- Functional → feature checking
- Non-functional → quality checking

Write "**next**" for next topic:

👉 **Testing Techniques (Black-box, White-box, Grey-box)**

next

Topic 6: Testing Techniques

Testing techniques means:

👉 Different ways/methods used to test software.

Main techniques:

1. Black-box Testing
2. White-box Testing
3. Grey-box Testing

These names sound difficult, but idea is simple.

1. Black-box Testing

Simple Meaning

Tester checks software from outside.

Tester:

- Does NOT see code
- Only checks input and output

Like using a closed black box.

Example

Login page testing:

- Enter username/password
- Check login success or failure

Tester does not care how code is written.

Focus

Checks:

- Features
- Functions
- Output

Real Life Example

Using TV remote:

- Press button
- TV works or not

You do not know internal circuits.

Same idea.

Advantages

- No coding knowledge needed
- Tests from user view

Short Definition

Black-box Testing tests software functionality without knowing internal code.

2. White-box Testing

Simple Meaning

Tester checks internal code and logic.

Tester knows:

- Program code
- Loops
- Conditions
- Logic

Usually done by programmers.

Example

Checking:

- Every if-else condition
- Every loop
- Every function path

Focus

Checks:

- Internal logic
- Code paths
- Statements

Real Life Example

Mechanic opens bike engine and checks inside parts.

Advantages

- Finds logic errors
- Improves code quality

Disadvantages

- Coding knowledge needed

Short Definition

White-box Testing tests internal code structure and program logic.

3. Grey-box Testing

Simple Meaning

Combination of:

- Black-box
- White-box

Tester has partial knowledge of code.

Example

Tester knows database structure but tests software like user.

Focus

Checks:

- Functionality
- Some internal logic

Short Definition

Grey-box Testing is testing with partial knowledge of internal code.

Easy Difference Table

Technique	Code Knowledge
Black-box	No knowledge
White-box	Full knowledge
Grey-box	Partial knowledge

Easy Real Example

Think about a car:

Technique	Example
Black-box	Driving car only
White-box	Opening engine fully
Grey-box	Know some engine details

Very Easy Memory Trick

Name	Meaning
Black-box	Cannot see inside
White-box	Can fully see inside
Grey-box	Can partially see inside

Very Short Summary

- Black-box → only input/output
- White-box → internal code testing
- Grey-box → partial code knowledge

Write "**next**" for next topic:

👉 **Test Case Design Techniques (Boundary Value Analysis & Equivalence Partitioning)**

next

Topic 7: Test Case Design Techniques

When testers test software, they create **test cases**.

But testing every possible input is very difficult.

So testers use smart techniques.

Main techniques in your syllabus:

1. Boundary Value Analysis (BVA)
2. Equivalence Partitioning (EP)

First Understand: What is Test Case?

A test case means:

👉 Steps and input used to test software.

Example:

Input	Expected Result
Correct password	Login successful

1. Boundary Value Analysis (BVA)

Simple Meaning

In this technique, testing is done on boundary/edge values.

Because errors mostly happen at limits.

Example

Suppose age allowed is:
18 to 60

Important boundary values:

- 17

- 18
- 19
- 59
- 60
- 61

Because bugs often occur near limits.

Why Use BVA?

Because programmers commonly make mistakes at:

- minimum value
 - maximum value
-

Real Life Example

Lift limit:

- Maximum 8 persons

Check:

- 7 persons
- 8 persons
- 9 persons

This is boundary testing.

Short Definition

Boundary Value Analysis is a testing technique where testing is done on minimum and maximum limit values.

Easy Memory Trick

Boundary = Border/Limit

So:

👉 Test near limits.

2. Equivalence Partitioning (EP)

Simple Meaning

Inputs are divided into groups/classes.

Instead of testing every value, tester tests one value from each group.

Example

Age allowed:
18 to 60

Groups:

Group	Valid/Invalid
Less than 18	Invalid
18 to 60	Valid
Greater than 60	Invalid

Now tester selects:

- 15
- 30
- 70

Only one value from each group.

Why Use EP?

Because testing all values wastes time.

So similar inputs are grouped together.

Real Life Example

Movie ticket age:

- Children
- Adults
- Senior citizens

One example tested from each category.

Short Definition

Equivalence Partitioning divides input data into valid and invalid groups for testing.

Difference Between BVA and EP

Boundary Value Analysis	Equivalence Partitioning
Tests edge values	Tests input groups
Focus on limits	Focus on categories
Example: 17,18,60,61	Example: one value from each group

Easy Combined Example

Marks allowed:

0 to 100

BVA values:

- -1
- 0
- 1
- 99
- 100
- 101

EP groups:

- Less than 0 → Invalid
- 0 to 100 → Valid

- Greater than 100 → Invalid
-

Very Short Summary

- BVA → test boundary values
- EP → divide inputs into groups

Write "next" for final topic:

👉 **Writing Effective Test Plans and Test Cases**

next

Topic 8: Writing Effective Test Plans and Test Cases

This is the final topic.

1. Test Plan

Simple Meaning

A Test Plan is a document that explains:

👉 How testing will be done.

It is like a roadmap for testing.

Test Plan Includes

- What to test
 - How to test
 - Who will test
 - Testing schedule
 - Tools needed
-

Real Life Example

Before cricket match:

- Team plan
- Players
- Strategy
- Timing

Same way, software testing also needs planning.

Main Parts of Test Plan

Part	Meaning
Objective	Purpose of testing
Scope	What will be tested
Resources	People/tools needed
Schedule	Testing time
Risks	Possible problems

Short Definition

A Test Plan is a document describing testing strategy, objectives, schedule, and resources.

2. Test Case

Simple Meaning

A Test Case is a set of steps used to check a feature.

It tells:

- What input to give
 - What output should come
-

Basic Test Case Format

Field	Meaning
Test Case ID	Unique number
Test Scenario	What is tested
Input	Data entered
Expected Result	Correct output
Actual Result	Real output
Status	Pass/Fail

Example Test Case

Login Page Testing

Field	Example
Test Case ID	TC_01
Scenario	Login with correct password
Input	Valid username/password
Expected Result	Login successful
Actual Result	Login successful
Status	Pass

Characteristics of Good Test Case

A good test case should be:

Quality	Meaning
Clear	Easy to understand
Simple	Not confusing
Complete	All steps included
Accurate	Correct expected result
Reusable	Can use again later

Tips for Writing Effective Test Cases

- ✓ Use simple language
- ✓ Write clear steps
- ✓ Mention expected output clearly
- ✓ Cover valid and invalid inputs
- ✓ Keep test cases organized

Easy Real Life Example

Testing calculator:

Input Expected Result

2 + 2 4

5 - 2 3

These are simple test cases.

Difference Between Test Plan and Test Case

Test Plan	Test Case
Overall testing strategy	Specific testing step
Big picture	Small individual check
Created first	Created after planning

Very Short Summary

- Test Plan → testing roadmap
- Test Case → individual testing steps
- Good test cases are clear and simple